

mova_auth · Login unificado

2.1 + 2.2 · Centralizar mova_auth + sesión server-side · Paso a paso con diagramas

El problema real (según el cliente)

Ya validan en backend (n8n vía tokeninfo + whitelist). El problema no es "falta de validación", es la fragmentación: Google + mova_auth con controles parciales por módulo, y JWT en localStorage.

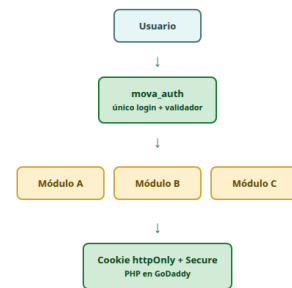
✗ HOY — Login fragmentado (problema)



La meta

mova_auth como único validador de sesión para todos los módulos M. Login único, token de sesión propio, cookie httpOnly en PHP (GoDaddy). Sin migrar de hosting.

✓ OBJETIVO — mova_auth centralizado



Fase A · Fase A — Inventario (no tocar código aún)

Día 1

Paso 1

Listar todos los módulos M

Abre cPanel → Administrador de archivos → public_html. Anota cada carpeta/módulo MOVA (MAESTRO, INGRESOS, EGRESOS, etc.) y su URL.

Estructura de carpetas en GoDaddy (cPanel)

```
public_html/
├─ mova_auth/          ← NUEVO (núcleo)
│   ├─ config.php
│   ├─ login.php
│   ├─ logout.php
│   ├─ validate.php    ← API validación
│   ├─ guard.php       ← include en módulos
│   └─ session.php
├─ modulo_maestro/
│   └─ index.php       ← require guard.php arriba
├─ modulo_ingresos/
└─ modulo_egresos/
```

👉 Crear tabla: Módulo | URL | ¿Cómo valida hoy? | ¿Usa localStorage?

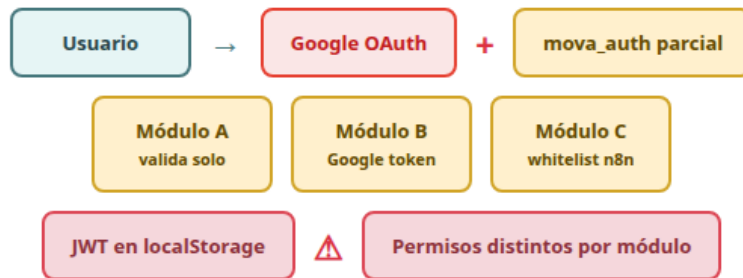
- ☐ Tabla de módulos completa
- ☐ URLs anotadas
- ☐ Responsable asignado por módulo

Paso 2

Documentar el flujo actual de cada módulo

Por cada módulo revisa el código (o pregunta al dev): ¿usa Google OAuth? ¿mova_auth? ¿JWT en localStorage? ¿llama a n8n?

✗ HOY — Login fragmentado (problema)



Cada módulo resuelve permisos por su cuenta → sesiones inconsistentes y riesgo de seguridad

👉 Marcar en rojo todo lo que NO pase por mova_auth

- ☐ Flujo actual dibujado o anotado
- ☐ JWT/localStorage identificados
- ☐ Endpoints n8n listados

Fase B · Fase B — Diseñar mova_auth (día 2-3)

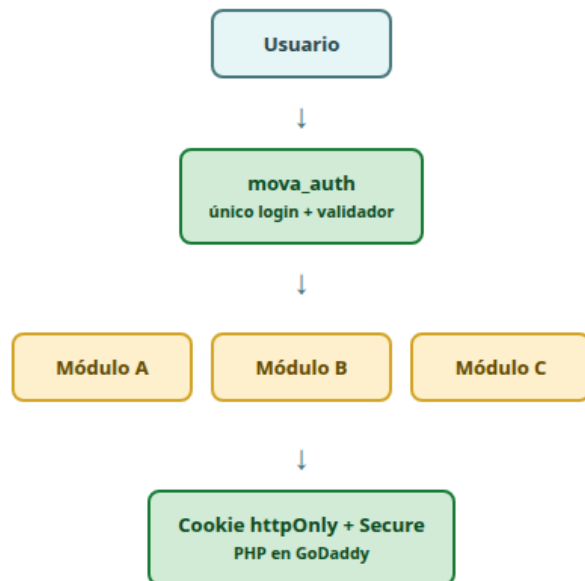
Días 2-3

Paso 3

Definir mova_auth como único validador

Acuerdo de equipo: ningún módulo valida permisos por su cuenta. Todos incluyen guard.php al inicio y consultan validate.php.

✓ OBJETIVO — mova_auth centralizado



Un solo login → un token de sesión → cada módulo solo consulta mova_auth

👉 Regla: si no pasó por mova_auth → redirect a login

- ☐ Regla escrita y compartida
- ☐ Lista de archivos a crear acordada

Paso 4

Crear carpeta mova_auth en GoDaddy

En cPanel → Administrador de archivos → public_html → Nueva carpeta: mova_auth. Sube los archivos base (ver Paso 5).

Estructura de carpetas en GoDaddy (cPanel)

```
public_html/
├─ mova_auth/      ← NUEVO (núcleo)
│  ├─ config.php
│  ├─ login.php
│  ├─ logout.php
│  ├─ validate.php ← API validación
│  ├─ guard.php    ← include en módulos
│  └─ session.php
├─ modulo_maestro/
│  └─ index.php    ← require guard.php arriba
├─ modulo_ingresos/
└─ modulo_egresos/
```

👉 **public_html/mova_auth/**

- ☐ Carpeta creada
- ☐ Permisos 755 en carpeta
- ☐ URL https://tudominio.cl/mova_auth/ responde

Paso 5

Subir archivos núcleo de mova_auth

Crear en mova_auth/: config.php (secretos), session.php (manejo sesión PHP), login.php (pantalla login), validate.php (API), guard.php (include módulos), logout.php.

Estructura de carpetas en GoDaddy (cPanel)

```
public_html/
├─ mova_auth/          ← NUEVO (núcleo)
│  ├─ config.php
│  ├─ login.php
│  ├─ logout.php
│  ├─ validate.php     ← API validación
│  ├─ guard.php        ← include en módulos
│  └─ session.php
├─ modulo_maestro/
│  └─ index.php        ← require guard.php arriba
├─ modulo_ingresos/
└─ modulo_egresos/
```

👉 guard.php es la pieza que unifica todos los módulos

```
// guard.php – incluir AL INICIO de cada módulo
<?php
require_once __DIR__ . '/../mova_auth/session.php';
if (!mova_session_valida()) {
    header('Location: /mova_auth/login.php?redirect=' .
    urlencode($_SERVER['REQUEST_URI']));
    exit;
}
```

- ☐ 6 archivos PHP creados
- ☐ config.php con secret fuera de Git público
- ☐ login.php abre en navegador

Fase C · Fase C — Login único y sesión (día 4-7)

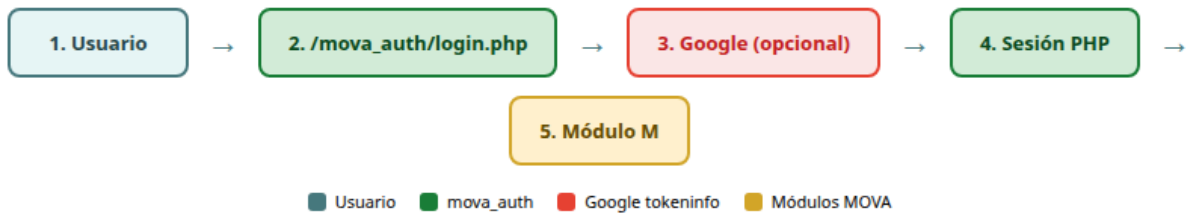
Días 4-7

Paso 6

Implementar login único

login.php: el usuario entra una sola vez. Si usas Google, valida con tokeninfo + whitelist (como hoy en n8n) pero el resultado se guarda en sesión PHP, no en localStorage.

Flujo de login (paso a paso)



👉 Flujo: login.php → (Google opcional) → crear sesión → redirect al módulo

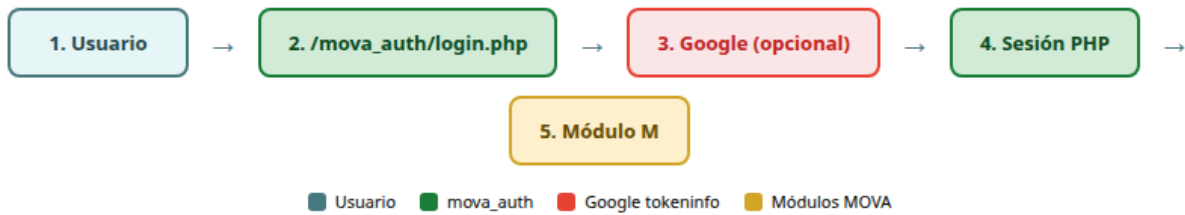
- ☐ Login funciona en navegador
- ☐ Redirect vuelve al módulo pedido
- ☐ Google tokeninfo probado

Paso 7

Token de sesión propio (no solo Google)

Tras login exitoso, genera un ID de sesión PHP (session_start) o token aleatorio guardado en servidor. No depender solo del token de Google en el cliente.

Flujo de login (paso a paso)



👉 **Cookie httpOnly + Secure — el cliente NO guarda JWT**

```
// session.php – emitir cookie segura
session_set_cookie_params([
  'lifetime' => 0,
  'path' => '/',
  'secure' => true,      // solo HTTPS
  'httponly' => true,    // JS no puede leerla
  'samesite' => 'Lax'
]);
session_start();
```

- ☐ Cookie visible en DevTools (HttpOnly ✓)
- ☐ Sin JWT en localStorage
- ☐ Sesión persiste al recargar

Paso 8

Endpoint validate.php

validate.php devuelve JSON { ok: true, usuario, permisos } si la sesión es válida. Los módulos HTML/JS pueden llamarlo con fetch (credentials: include) para AJAX.

Cómo valida cada módulo



👉 Cada petición AJAX del módulo → validate.php primero

- ☐ validate.php responde 200 con sesión
- ☐ Responde 401 sin sesión
- ☐ Permisos correctos por rol

Paso 9

Migrar módulos uno por uno

Orden sugerido: 1) módulo menos crítico (sandbox), 2) resto. En cada index.php agrega require guard.php arriba. Elimina validación duplicada del módulo.

Cómo valida cada módulo



👉 Un módulo a la vez — probar antes de pasar al siguiente

- ☐ Módulo 1 migrado y probado
- ☐ Módulo 2 migrado
- ☐ Todos los M usan guard.php

Fase D · Fase D — Limpieza y pruebas (día 8-10)

Días 8-10

Paso 10

Retirar JWT de localStorage

Busca en cada módulo: `localStorage.setItem`, `localStorage.getItem` con `token/jwt`. Elimina y reemplaza por sesión PHP (el navegador envía la cookie sola).

👉 DevTools → Application → Local Storage → debe quedar vacío de tokens

- ☐ localStorage sin JWT
- ☐ Código viejo comentado/eliminado
- ☐ Sin regresiones en módulos

Paso 11

Pruebas por módulo

Por cada M: (1) sin login → redirect login, (2) con login → entra, (3) logout → no puede volver, (4) otra pestaña incógnito → pide login.

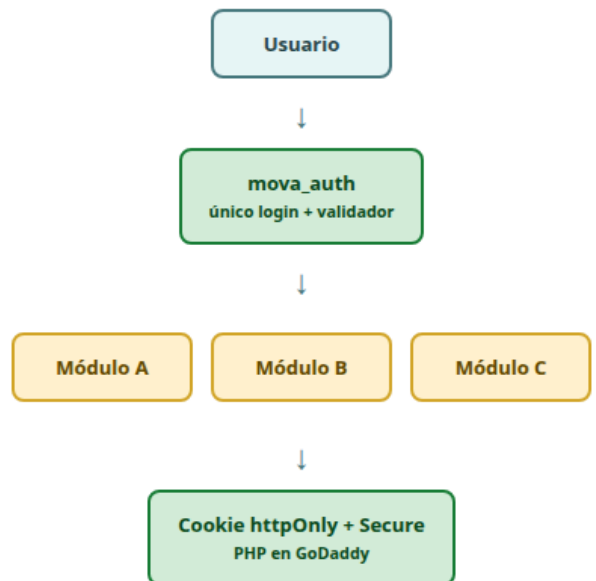
👉 Probar en Chrome normal + ventana incógnito

- ☐ 4 pruebas OK por módulo
- ☐ Logout funciona
- ☐ Sesión expira según config

Documentar y cerrar hito 2.1 + 2.2

Actualiza ficha MOVA: URL login, dónde está config.php, quién tiene acceso cPanel. Marca en Gantt como completado.

✓ OBJETIVO — mova_auth centralizado



Un solo login → un token de sesión → cada módulo solo consulta mova_auth

👉 Entregable: mova_auth operativo + todos los M centralizados

- ☐ Documentación actualizada
- ☐ Equipo capacitado
- ☐ Gantt hito 2.1 y 2.2 cerrados

Errores comunes y solución

Error	Qué hacer
Bucle de redirect infinito	login.php no debe incluir guard.php. Excluir /mova_auth/ del guard.
Cookie no se guarda	Verificar HTTPS activo (Cloudflare SSL Full), secure => true requiere HTTPS.
Módulo sigue pidiendo login	Misma cookie path=/ y dominio. Revisar que guard.php apunte a la misma carpeta mova_auth.
AJAX devuelve 401	fetch debe usar credentials: "include" para enviar la cookie.

Error	Qué hacer
Google OK pero sesión no persiste	session_start() antes de cualquier output. Revisar session.save_path en PHP.

No hacer

- No migrar de GoDaddy ahora (movimiento lateral)
- No validar permisos dentro de cada módulo
- No guardar tokens en localStorage
- No migrar todos los módulos el mismo día sin probar

Checklist final — hito 2.1 + 2.2

- ☐ mova_auth es el único punto de login
- ☐ Todos los módulos M incluyen guard.php
- ☐ Cookie httpOnly + Secure activa
- ☐ Sin JWT en localStorage
- ☐ validate.php operativo
- ☐ Pruebas OK en todos los módulos
- ☐ Documentación en *ficha MOVA*